Binary Logistic Regression Case Study: Marketing Campaign Response Model

Background: The large skin-clinic in the city of “XXX” offers variety of products and services to customers. The marketing campaign was launched for introducing a new product.

The data shows response to the marketing campaign alongwith demographic and transaction data of customers.

The objective is to identify factors driving response to the marketing campaign.

Import datasets and create masterdata

```
import pandas as pd
import os
os.chdir("D:/SANKHYA/Purchase_Response BLR")
purchase_data1 = pd.read_csv('Purchase Data 1.csv')
purchase_data2 = pd.read_csv('Purchase Data 2.csv')
response_data = pd.read_csv('Response Data.csv')
masterdata=pd.merge(response_data,purchase_data1,how='left')
masterdata=pd.merge(masterdata,purchase_data2,how='left')
```

Convert numerically coded categorical variables into “categories”

```
masterdata['Age'] = masterdata['Age'].astype('category')
masterdata['Gender'] = masterdata['Gender'].astype('category')
masterdata['MS'] = masterdata['MS'].astype('category')
masterdata['Pre_Month'] = masterdata['Pre_Month'].astype('category')
```

Display first 6 rows

```
masterdata.head(6)
```

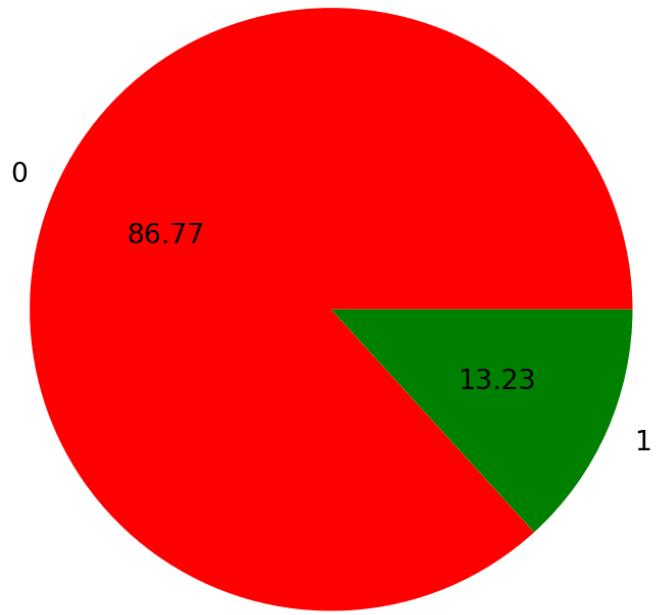
```
   Custid Age Gender MS  ...  N_Service BillAmt_1  BillAmt_2  BillAmt_3
0       1   2      1  2  ...         24     12.34      13.26       5.88
1       2   2      2  1  ...         29     18.65       2.12       5.13
2       3   1      1  2  ...         21      7.22       3.31       3.65
3       4   2      1  1  ...         22      6.15       2.95       2.34
4       5   2      2  1  ...         35     20.64       2.67       4.07
5       6   1      2  1  ...         31     12.44       3.06       2.57

[6 rows x 11 columns]
```

Data Description

Column name	**Column description**
Custid	Customer Identification Code
Age	Age group(1:<32,2:32-48,3:>48
Gender	Gender(1:Female,2:Male)
MS	Marital Status(1: Not Married,2:Married)
BillAmt_1,BillAmt_2,BillAmt_3	Bill Amounts (USD) for Last Three Purchases
Pre_Month	Purchase in Previous Week(1: Purchased in Previous Week,2: No purchase in Previous Week)
N_Products	Number of unique products purchased in One Year
N_Service	Number of unique services purchased in one year)

Visualize Campaign Response Rate

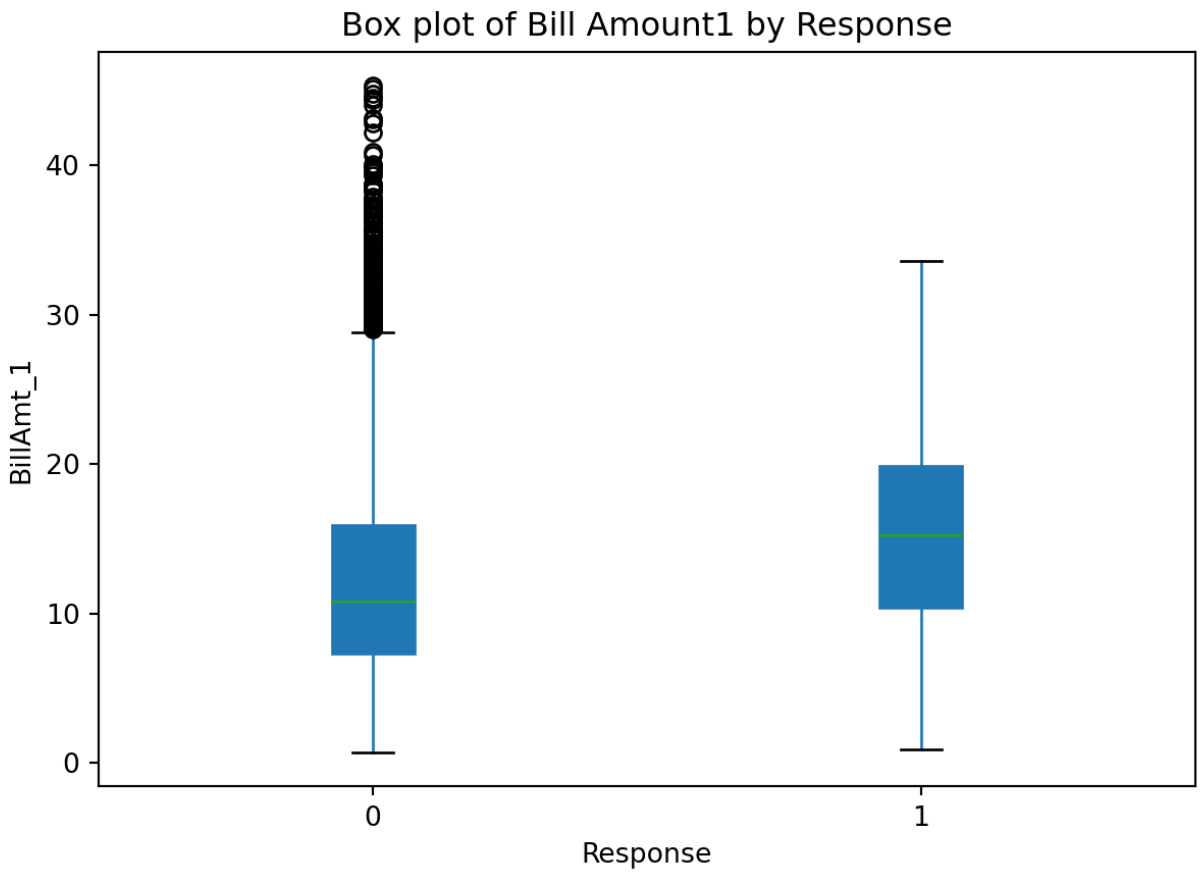
```
import numpy as np
import matplotlib.pyplot as plt

response = pd.DataFrame(masterdata['Response'].value_counts()).reset_index()
response = response.rename(columns={'index': 'Response', 'Response': 'Count'})

y = np.array(response['Count'])
mylabels = response['Response']

plt.pie(y, labels = mylabels,autopct='%.2f',colors = ['red','green']);
plt.show()
```



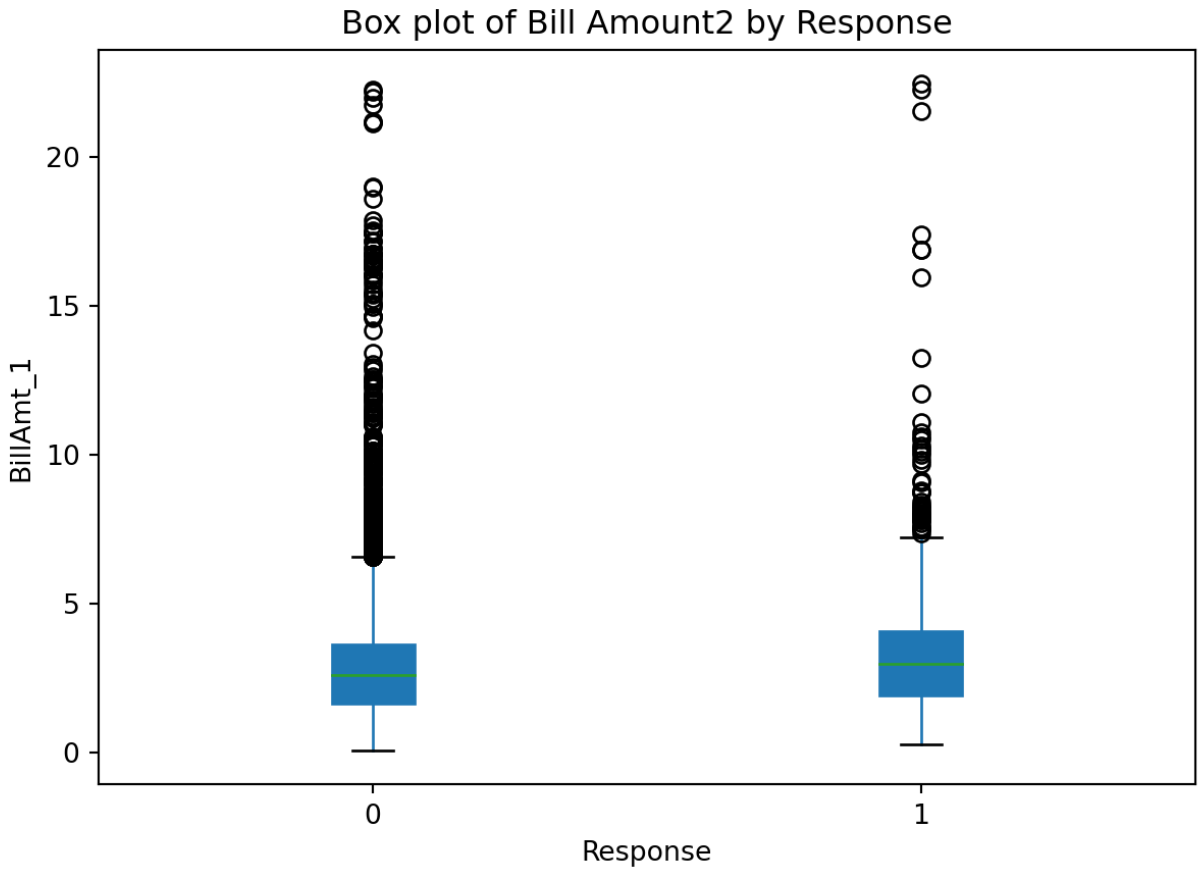
Comment: Approximately 13% of customers responded positively to the campaign

Box plot of Bill Amounts by Response

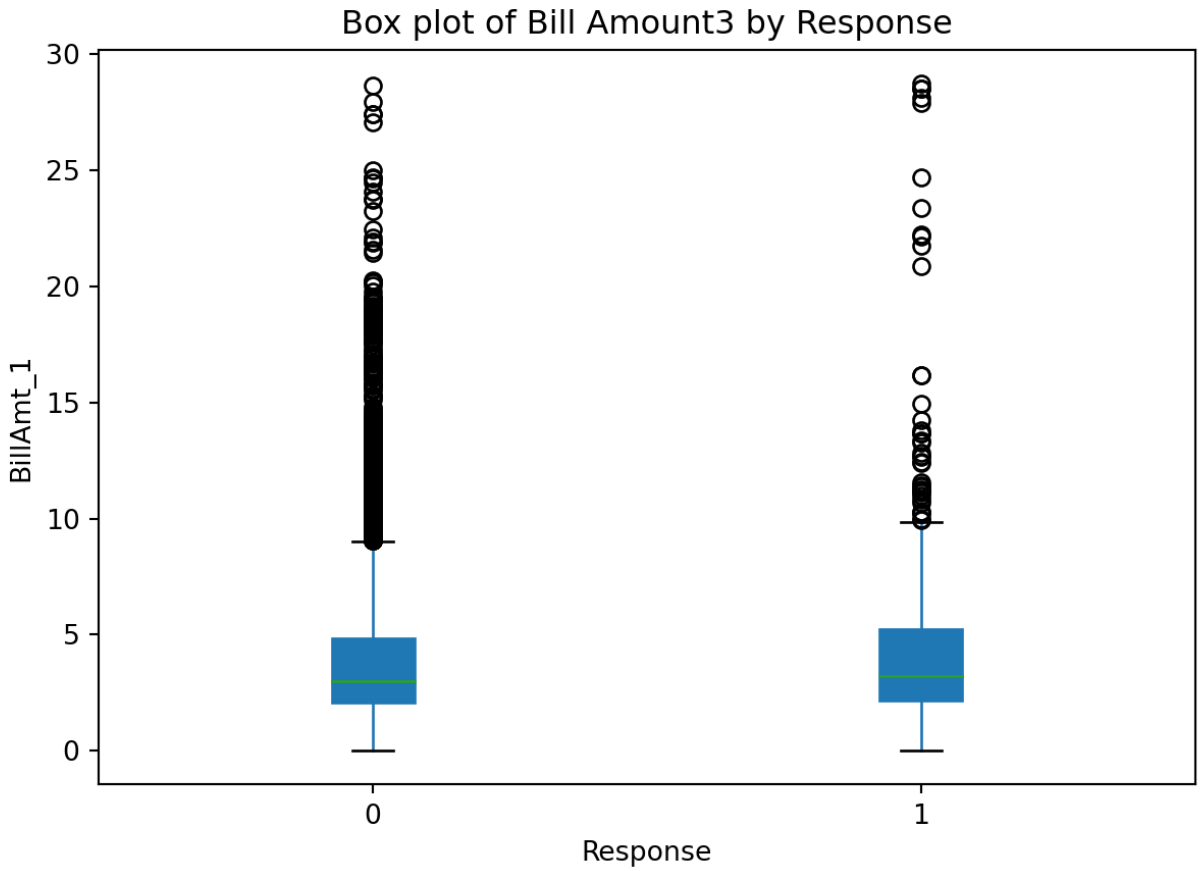
```
masterdata.boxplot(column='BillAmt_1', by='Response', grid=False, patch_artist
=True);plt.title('Box plot of Bill Amount1 by Response');plt.suptitle('');pl
t.ylabel('BillAmt_1')
plt.show();
```




```
masterdata.boxplot(column='BillAmt_2', by='Response', grid=False, patch_artist
=True);plt.title('Box plot of Bill Amount2 by Response');plt.suptitle('');pl
t.ylabel('BillAmt_1')
plt.show();
```




```
masterdata.boxplot(column='BillAmt_3', by='Response', grid=False, patch_artist
=True);plt.title('Box plot of Bill Amount3 by Response');plt.suptitle('');pl
t.ylabel('BillAmt_1')
plt.show();
```



Comment: Higher bill amounts in last 2 transactions are observed for customers with positive response

Developing Campaign Response Model using Binary Logistic Regression

Splitting original data into training and testing data sets

```
import numpy as np
from sklearn.model_selection import train_test_split

X = pd.DataFrame(np.c_[masterdata.drop(['Response','Custid'],1)])

Y = masterdata['Response']

from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.3, ran
dom_state=5)
```

Binary Logistic Regression using LogisticRegression function from sklearn library

```
from sklearn.linear_model import LogisticRegression

logreg = LogisticRegression(max_iter=200)

logreg.fit(X_train,Y_train)
```

```
LogisticRegression(max_iter=200)
```

Display model coefficients

```
logreg.coef_
```

```
array([[-0.04461695,  0.08753498, -0.16148322,  0.03835953,  0.14571507,
         0.0363377 ,  0.03507174,  0.17521301,  0.01371423]])
```

Comment: In order to get p values from logistic regression, we can use logit function from statsmodels library

Check for multicollinearity using vif

```
from patsy import dmatrices
from statsmodels.stats.outliers_influence import variance_inflation_factor

y1, X1 = dmatrices('Response~Age+Gender+MS+Pre_Month+N_Products+N_Service+Bill
Amt_1+BillAmt_2+BillAmt_3', data=masterdata, return_type="dataframe")

vif = pd.Series([variance_inflation_factor(X1.values, i)for i in range(X1.shap
e[1])],index=X1.columns)
vif
```

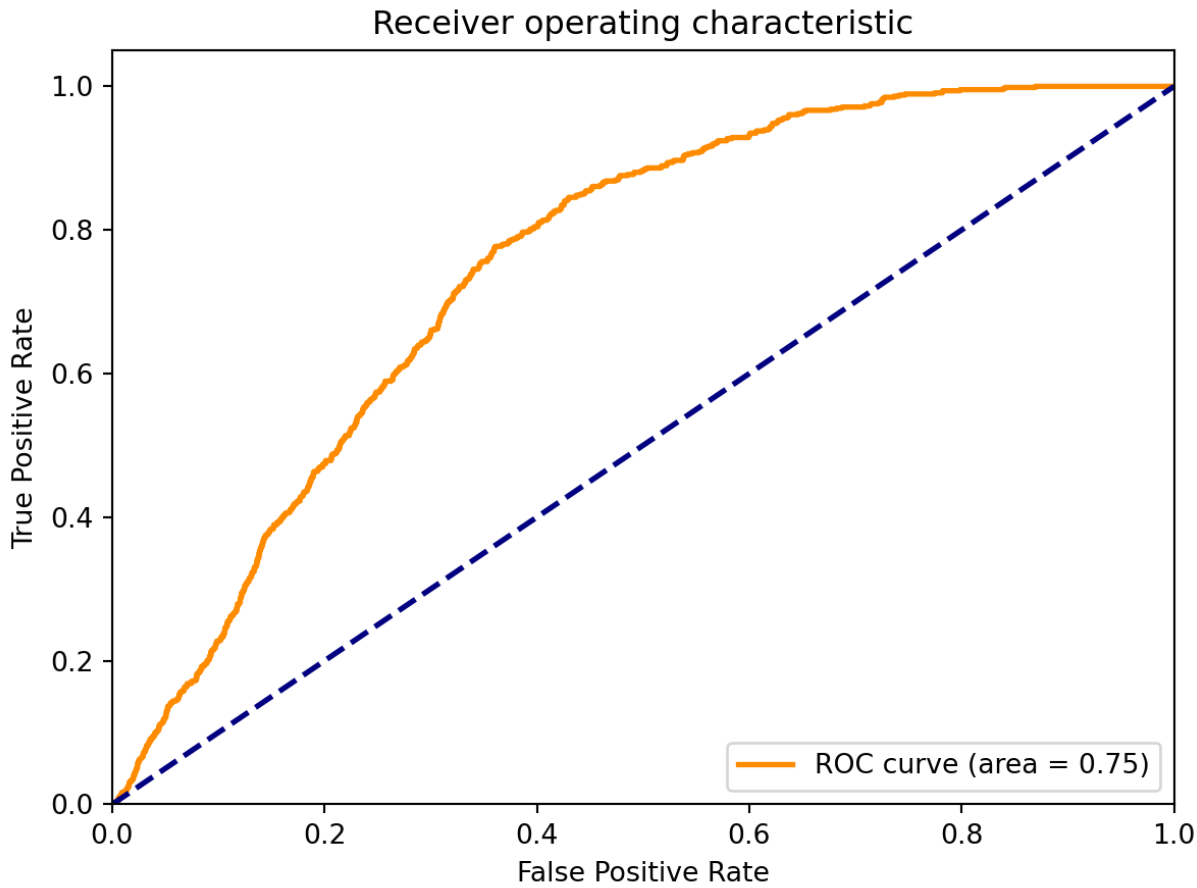
```
Intercept         41.887248
Age[T.2]           1.320898
Age[T.3]           1.319500
Gender[T.2]        1.001581
MS[T.2]            1.001546
Pre_Month[T.2]     1.001391
N_Products         1.507631
N_Service          1.138544
BillAmt_1          1.793005
BillAmt_2          1.688835
BillAmt_3          2.111681
dtype: float64
```

Comment: It is observed that no variable has high vif

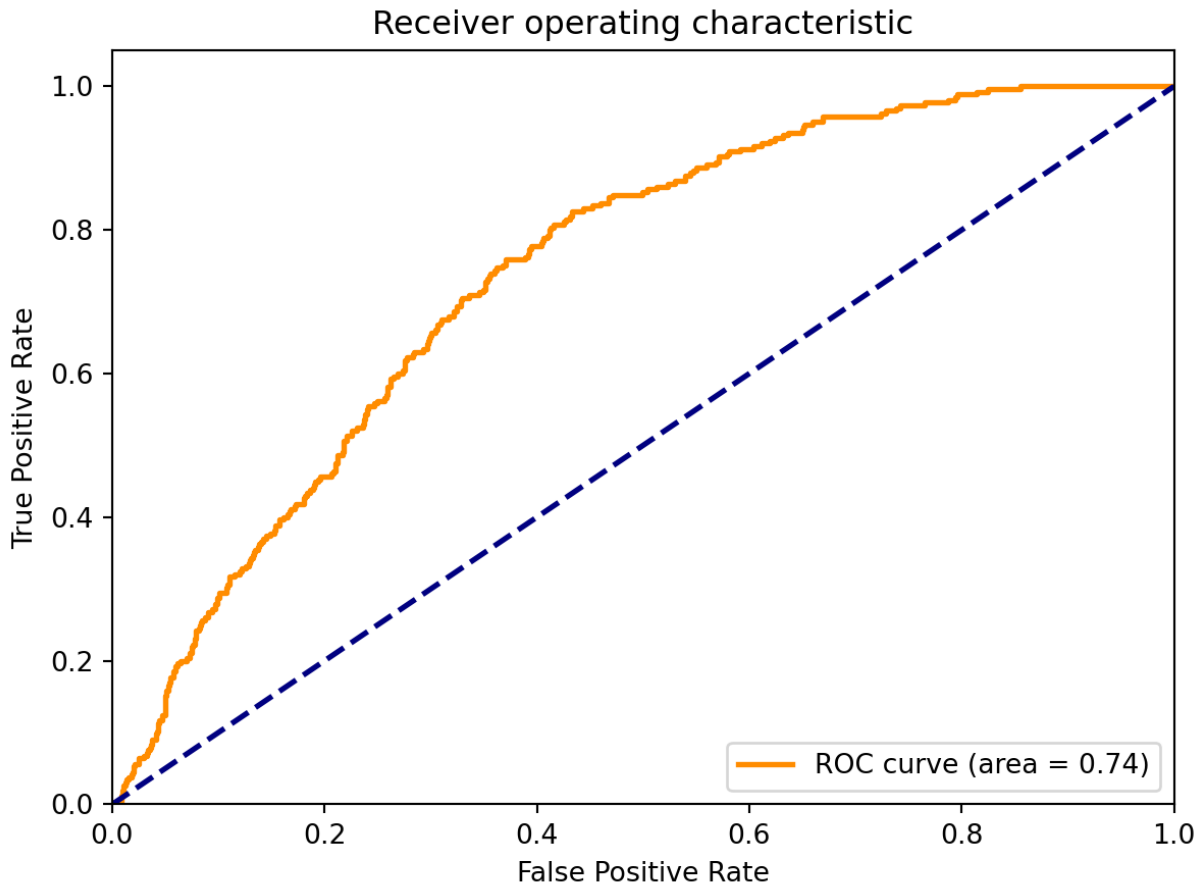
Hence the problem of multicollinearity does not exist

Evaluate model performance by ROC Curve

```
from sklearn import metrics
from sklearn.metrics import roc_curve, auc

y_pred_probtrain = logreg.predict_proba(X_train)[::,1]
fpr, tpr, thresholds =roc_curve(Y_train,y_pred_probtrain)
roc_auc = auc(fpr, tpr)

ruc_auc = auc(fpr,tpr)

plt.figure();
lw = 2
plt.plot(fpr, tpr, color='darkorange',lw=lw, label='ROC curve (area = %0.2f)'
% ruc_auc);
plt.plot([0, 1], [0, 1], color='navy', lw=lw, linestyle='--');
plt.xlim([0.0, 1.0]);plt.ylim([0.0, 1.05]);
plt.xlabel('False Positive Rate');plt.ylabel('True Positive Rate');
plt.title('Receiver operating characteristic');plt.legend(loc="lower right");
plt.show()
```




```
y_pred_probtest = logreg.predict_proba(X_test)[::,1]
fpr, tpr, thresholds =roc_curve(Y_test,y_pred_probtest)
roc_auc = auc(fpr, tpr)

ruc_auc = auc(fpr,tpr)

plt.figure();
lw = 2
plt.plot(fpr, tpr, color='darkorange',lw=lw, label='ROC curve (area = %0.2f)'
% ruc_auc);
plt.plot([0, 1], [0, 1], color='navy', lw=lw, linestyle='--');
plt.xlim([0.0, 1.0]);plt.ylim([0.0, 1.05]);
plt.xlabel('False Positive Rate');plt.ylabel('True Positive Rate');
plt.title('Receiver operating characteristic');plt.legend(loc="lower right");
plt.show()
```



Model Validation: Holdout Method using Confusion matrix

Find optimum threshold for classification by maximizing sensitivity + specificity

```
fpr, tpr, thresholds =roc_curve(Y_train,y_pred_probtrain)
roc_auc = auc(fpr, tpr)

i = np.arange(len(tpr))
roc = pd.DataFrame({'fpr' : pd.Series(fpr, index=i),'tpr' : pd.Series(tpr, ind
ex = i), '1-fpr' : pd.Series(1-fpr, index = i), 'tf' : pd.Series(tpr - (1-fp
r), index = i), 'thresholds' : pd.Series(thresholds, index = i)})

optimal_idx = np.argmax(tpr - fpr)
optimal_threshold = roc.iloc[optimal_idx][4]
optimal_threshold
```

```
0.13473353416735645
```

Confusion matrix for train and test data

```
predicted_values1=logreg.predict_proba(X_train)[::,1]
threshold=optimal_threshold
predicted_class1=np.zeros(predicted_values1.shape)
predicted_class1[predicted_values1>threshold]=1

from sklearn.metrics import classification_report
print(classification_report(Y_train,predicted_class1))
```

```
              precision    recall  f1-score   support

           0       0.95      0.64      0.76      4233
           1       0.25      0.78      0.38       660

    accuracy                           0.66      4893
   macro avg       0.60      0.71      0.57      4893
weighted avg       0.85      0.66      0.71      4893
```

```
predicted_values1=logreg.predict_proba(X_test)[::,1]
threshold=optimal_threshold
predicted_class1=np.zeros(predicted_values1.shape)
predicted_class1[predicted_values1>threshold]=1

print(classification_report(Y_test,predicted_class1))
```

```
              precision    recall  f1-score   support

           0       0.95      0.63      0.75      1832
           1       0.23      0.76      0.35       265

    accuracy                           0.64      2097
   macro avg       0.59      0.69      0.55      2097
weighted avg       0.86      0.64      0.70      2097
```

K-fold cross validation

```
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import cohen_kappa_score

predicted_prob = cross_val_predict(logreg, X, Y, cv=4, method='predict_proba')
threshold=optimal_threshold
predicted = predicted_prob[:,1]
predicted_class1=np.zeros(predicted.shape)
predicted_class1[predicted>threshold]=1

print(classification_report(Y,predicted_class1))
```

```
              precision    recall  f1-score   support

           0       0.94      0.64      0.77      6065
           1       0.24      0.75      0.37       925

    accuracy                           0.66      6990
   macro avg       0.59      0.70      0.57      6990
weighted avg       0.85      0.66      0.71      6990
```

Comment: The Area Under ROC Curve (AUC) for train and test data indicate that the model is stable and performed well on test data. The sensitivity value using optimum threshold is 76% for test data. There is a scope to improve specificity and overall accuracy if more variables are available such as location of the customer, months since first transaction etc.

In the absence of additional variables, the model can be implemented for decision making in next planned campaign.